

```
"""
```

Telegram bot for tracking sportsbook odds (soccer, baseball, table tennis)
and manually-logged free-bet promos.

You place all bets yourself – this bot only surfaces information.

Commands:

```
/start                - intro
/sports               - list currently active sport_keys from the API
/odds <sport_key>     - show best current odds for upcoming matches
/watch <sport_key> <event_id> - start tracking a match for line movement
/watchlist           - show what you're watching
/unwatch <event_id>   - stop tracking a match
/addfreebet <bookmaker> | <description> | <amount> | <YYYY-MM-DD expiry>
/freebets             - list your active free bets
/usedfreebet <id>    - mark a free bet as used
```

```
"""
```

```
import logging
```

```
import os
```

```
from dotenv import load_dotenv
```

```
from telegram import Update
```

```
from telegram.ext import Application, CommandHandler, ContextTypes
```

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler
```

```
import storage
```

```
from odds_client import OddsClient, OddsAPIError
```

```
load_dotenv()
```

```
TELEGRAM_BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
```

```
TELEGRAM_CHAT_ID = int(os.environ["TELEGRAM_CHAT_ID"])
```

```
ODDS_API_KEY = os.environ["ODDS_API_KEY"]
```

```
POLL_INTERVAL_MINUTES = int(os.getenv("POLL_INTERVAL_MINUTES", "60"))
```

```
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
```

```
log = logging.getLogger("oddsbot")
```

```
odds_client = OddsClient(ODDS_API_KEY)
```

```
def _authorized(update: Update) -> bool:
```

```
    return update.effective_chat.id == TELEGRAM_CHAT_ID
```

```

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    await update.message.reply_text(
        "Odds tracker bot online.\n\n"
        "This bot surfaces odds and free-bet reminders. You place every bet yours\n\n"
        "Try:\n"
        "/sports - list active sport keys\n"
        "/odds soccer_epl - best current odds\n"
        "/watch soccer_epl <event_id> - track line movement\n"
        "/freebets - your tracked free bets"
    )

```

```

async def sports_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    try:
        sports = odds_client.list_sports()
    except OddsAPIError as e:
        await update.message.reply_text(f"Error fetching sports: {e}")
        return

    wanted_groups = {"Soccer", "Baseball", "Table Tennis"}
    filtered = [s for s in sports if s.get("group") in wanted_groups]
    if not filtered:
        filtered = sports[:25] # fallback: just show something

    lines = [f"`{s['key']}` - {s['title']} ({s['group']})" for s in filtered[:40]]
    await update.message.reply_text(
        "Active sport keys (soccer/baseball/table tennis where available):\n\n"
        + "\n".join(lines),
        parse_mode="Markdown",
    )

```

```

async def odds_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    if not context.args:
        await update.message.reply_text("Usage: /odds <sport_key> (see /sports f
        return

    sport_key = context.args[0]
    try:

```

```

        events = odds_client.get_odds(sport_key)
    except OddsAPIError as e:
        await update.message.reply_text(f"Error fetching odds: {e}")
        return

    if not events:
        await update.message.reply_text("No upcoming events found for that sport")
        return

    lines = []
    for event in events[:8]:
        home, away = event.get("home_team"), event.get("away_team")
        best = odds_client.best_prices(event)
        lines.append(f"\n*{home} vs {away}*  (`{event['id']}`)")
        for name, info in best.items():
            lines.append(f"  {name}: {info['price']}  ({info['bookmaker']})")

    await update.message.reply_text("\n".join(lines), parse_mode="Markdown")

async def watch_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    if len(context.args) < 2:
        await update.message.reply_text("Usage: /watch <sport_key> <event_id>")
        return

    sport_key, event_id = context.args[0], context.args[1]
    try:
        event = odds_client.get_event_odds(sport_key, event_id)
    except OddsAPIError as e:
        await update.message.reply_text(f"Couldn't find that event: {e}")
        return

    label = f"{event.get('home_team')} vs {event.get('away_team')}"
    storage.add_watch(sport_key, event_id, label)

    best = odds_client.best_prices(event)
    for name, info in best.items():
        storage.save_snapshot(event_id, name, info["price"], info["bookmaker"])

    await update.message.reply_text(f"Now watching: {label}\nYou'll get alerts on")

async def watchlist_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return

```

```

watches = storage.list_watches()
if not watches:
    await update.message.reply_text("Not watching any matches yet. Use /watch")
    return
lines = [f"`{w['event_id']}` - {w['event_label']} ({w['sport_key']})" for w in watches]
await update.message.reply_text("\n".join(lines), parse_mode="Markdown")

async def unwatch_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    if not context.args:
        await update.message.reply_text("Usage: /unwatch <event_id>")
        return
    storage.remove_watch(context.args[0])
    await update.message.reply_text("Removed from watchlist.")

async def addfreebet_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    raw = " ".join(context.args)
    parts = [p.strip() for p in raw.split("|")]
    if len(parts) < 2:
        await update.message.reply_text(
            "Usage: /addfreebet <bookmaker> | <description> | <amount optional> |"
        )
        return

    bookmaker, description = parts[0], parts[1]
    amount = parts[2] if len(parts) > 2 else ""
    expires_at = None
    if len(parts) > 3 and parts[3]:
        import datetime
        try:
            expires_at = int(datetime.datetime.strptime(parts[3], "%Y-%m-%d").timestamp())
        except ValueError:
            await update.message.reply_text("Expiry date must be YYYY-MM-DD. Save")

    storage.add_free_bet(bookmaker, description, amount, expires_at)
    await update.message.reply_text(f"Saved free bet: {bookmaker} - {description}")

async def freebets_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    bets = storage.list_free_bets()

```

```

if not bets:
    await update.message.reply_text("No active free bets tracked. Add one wit
    return

import datetime
lines = []
for b in bets:
    exp = (
        datetime.datetime.fromtimestamp(b["expires_at"]).strftime("%Y-%m-%d")
        if b["expires_at"] else "no expiry"
    )
    lines.append(f"#{b['id']} {b['bookmaker']}: {b['description']} ({b['amoun
await update.message.reply_text("\n".join(lines))

async def usedfreebet_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    if not context.args:
        await update.message.reply_text("Usage: /usedfreebet <id>")
        return
    storage.mark_free_bet_used(int(context.args[0]))
    await update.message.reply_text("Marked as used.")

# --- Scheduled jobs ---

async def check_line_movement(app: Application):
    watches = storage.list_watches()
    for w in watches:
        try:
            event = odds_client.get_event_odds(w["sport_key"], w["event_id"])
        except OddsAPIError as e:
            log.warning(f"Skipping {w['event_id']}: {e}")
            continue

    best = odds_client.best_prices(event)
    for name, info in best.items():
        prev = storage.last_snapshot(w["event_id"], name)
        storage.save_snapshot(w["event_id"], name, info["price"], info["bookm
        if prev and abs(prev["price"] - info["price"]) >= 0.05:
            direction = "shortened" if info["price"] < prev["price"] else "dr
            await app.bot.send_message(
                chat_id=TELEGRAM_CHAT_ID,
                text=(
                    f"Line movement: {w['event_label']}\n"
                    f"{name} {direction}: {prev['price']} -> {info['price']}"

```

```

        f"({info['bookmaker']})"
    ),
)

async def check_expiring_free_bets(app: Application):
    soon = storage.expiring_soon(within_seconds=48 * 3600)
    for b in soon:
        await app.bot.send_message(
            chat_id=TELEGRAM_CHAT_ID,
            text=f"Free bet expiring soon: {b['bookmaker']} - {b['description']}"
        )

def main():
    storage.init_db()
    app = Application.builder().token(TELEGRAM_BOT_TOKEN).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("sports", sports_cmd))
    app.add_handler(CommandHandler("odds", odds_cmd))
    app.add_handler(CommandHandler("watch", watch_cmd))
    app.add_handler(CommandHandler("watchlist", watchlist_cmd))
    app.add_handler(CommandHandler("unwatch", unwatch_cmd))
    app.add_handler(CommandHandler("addfreebet", addfreebet_cmd))
    app.add_handler(CommandHandler("freebets", freebets_cmd))
    app.add_handler(CommandHandler("usedfreebet", usedfreebet_cmd))

    scheduler = AsyncIOScheduler()
    scheduler.add_job(check_line_movement, "interval", minutes=POLL_INTERVAL_MINU
    scheduler.add_job(check_expiring_free_bets, "interval", hours=12, args=[app])
    scheduler.start()

    log.info("Bot starting...")
    app.run_polling()

if __name__ == "__main__":
    main()

```